

# Media Player Feed System: Complete Rebuild

Fresh build. No code from Clubhouse page. New algorithm, new queries, new hooks, new types.

---

## Section 1: Data Types

### `types/media.ts` — Updated

Add these types alongside the existing `MediaItem` and `FeedPost` types. Don't remove the existing types — extend them.

### New Types

```
/** Feed tab modes */
export type FeedTab = 'suggested' | 'friends';

/** Review data attached to a post */
export interface ReviewData {
  reviewId: string;
  courseId: string;
  courseName: string;
  courseImageUrl: string | null;
  rating: number;           // 1-5 star rating from course_ratings
}

/** Creator relationship to current user */
export type CreatorRelation = 'friend' | 'following' | 'none';
```

### Updated `FeedPost` Type

Replace the existing `FeedPost` with this expanded version:

```
export interface FeedPost {
  id: string;
  userId: string;
  actorType: 'personal' | 'business';
  actorId: string;           // = userId for personal, businessId for business
```

```

// Creator info
username: string;
displayName: string;
avatarUrl: string;
isVerified: boolean;           // Real verified status from DB — NO Math.random
creatorRelation: CreatorRelation;

// Content
caption: string;
mediaItems: MediaItem[];
createdAt: string;             // ISO timestamp

// Engagement (real counts from DB)
likeCount: number;
commentCount: number;
shareCount: number;

// Review data (null for regular posts)
review: ReviewData | null;
isReview: boolean;

// Interaction state (for current user)
isLikedByMe: boolean;
isFollowedByMe: boolean;
}

```

---

## Section 2: Supabase RPC Function — Engagement-Scored Feed

### `get_suggested_feed` — Postgres Function

This is the heart of the algorithm. It runs server-side in Postgres, returning engagement-scored posts ready to display. No client-side sorting needed.

**Create this as a Supabase SQL migration:**

```

CREATE OR REPLACE FUNCTION get_suggested_feed(
  p_user_id UUID,
  p_page_size INT DEFAULT 10,
  p_cursor TIMESTAMPTZ DEFAULT NULL,    -- created_at of last seen post

```

```

    p_seen_post_ids UUID[] DEFAULT '{}' -- posts already shown this session
)
RETURNS TABLE (
    -- Post fields
    post_id UUID,
    post_content TEXT,
    post_created_at TIMESTAMPTZ,
    post_user_id UUID,
    post_actor_type TEXT,
    post_actor_id UUID,
    post_status TEXT,
    source_review_id UUID,

    -- Media fields (first media item - we'll fetch all media separately if multi-m
    media_id UUID,
    media_type TEXT,
    media_url TEXT,
    poster_url TEXT,
    stream_id TEXT,
    duration_seconds NUMERIC,
    width INT,
    height INT,
    display_order INT,

    -- Creator profile
    creator_username TEXT,
    creator_display_name TEXT,
    creator_avatar_url TEXT,
    creator_is_verified BOOLEAN,

    -- Business profile (null for personal posts)
    business_name TEXT,
    business_logo_url TEXT,
    business_is_verified BOOLEAN,

    -- Engagement counts
    like_count BIGINT,
    comment_count BIGINT,
    share_count BIGINT,

    -- Review data (null for non-reviews)
    review_rating NUMERIC,
    review_course_id UUID,
    review_course_name TEXT,
    review_course_image TEXT,

    -- Relationship to requesting user

```

```

creator_relation TEXT,          -- 'friend', 'following', 'none'
is_liked_by_me BOOLEAN,
is_followed_by_me BOOLEAN,

-- Computed score (for debugging/transparency)
engagement_score NUMERIC
)
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
    v_recency_halflife_hours NUMERIC := 24;    -- Score halves every 24 hours
    v_max_post_age_days INT := 14;             -- Don't show posts older than 14 day
    v_max_duration_seconds INT := 180;         -- Shorts = under 3 minutes
    v_creator_cap INT := 2;                   -- Max 2 posts per creator per page
    v_fetch_multiplier INT := 3;              -- Fetch 3x page_size to allow filter
BEGIN
    RETURN QUERY
    WITH

    -- Step 1: Get the user's social graph
    my_friends AS (
        SELECT friend_id AS uid FROM user_friends
        WHERE user_id = p_user_id AND status = 'accepted'
        UNION
        SELECT user_id AS uid FROM user_friends
        WHERE friend_id = p_user_id AND status = 'accepted'
    ),
    my_follows AS (
        SELECT following_id AS uid FROM user_follows
        WHERE follower_id = p_user_id
    ),
    my_business_follows AS (
        SELECT business_id AS bid FROM business_follows
        WHERE follower_id = p_user_id
    ),
    blocked_users AS (
        SELECT blocked_user_id AS uid FROM user_blocks
        WHERE blocker_user_id = p_user_id
        UNION
        SELECT blocker_user_id AS uid FROM user_blocks
        WHERE blocked_user_id = p_user_id
    ),

    -- Step 2: Fetch candidate posts with media
    candidates AS (
        SELECT

```

```

p.id AS p_id,
p.content AS p_content,
p.created_at AS p_created_at,
p.user_id AS p_user_id,
p.actor_type AS p_actor_type,
p.actor_id AS p_actor_id,
p.status AS p_status,
p.source_review_id AS p_source_review_id,

pm.id AS pm_id,
pm.media_type AS pm_media_type,
pm.media_url AS pm_media_url,
pm.poster_url AS pm_poster_url,
pm.stream_id AS pm_stream_id,
pm.duration_seconds AS pm_duration,
pm.width AS pm_width,
pm.height AS pm_height,
pm.display_order AS pm_display_order,

-- Engagement counts
COALESCE(lc.cnt, 0) AS p_like_count,
COALESCE(cc.cnt, 0) AS p_comment_count,
COALESCE(sc.cnt, 0) AS p_share_count,

-- Relationship
CASE
    WHEN mf.uid IS NOT NULL THEN 'friend'
    WHEN mfl.uid IS NOT NULL THEN 'following'
    WHEN mbf.bid IS NOT NULL AND p.actor_type = 'business' THEN 'following'
    ELSE 'none'
END AS p_relation,

-- Is the current user liking/following this creator
CASE WHEN ml.post_id IS NOT NULL THEN TRUE ELSE FALSE END AS p_liked_by_me,
CASE
    WHEN mfl.uid IS NOT NULL THEN TRUE
    WHEN mbf.bid IS NOT NULL AND p.actor_type = 'business' THEN TRUE
    ELSE FALSE
END AS p_followed_by_me,

-- Review data
cr.rating AS p_review_rating,
gc.id AS p_review_course_id,
gc.name AS p_review_course_name,
gc.image_url AS p_review_course_image,

-- Creator profile

```

```

up.username AS p_creator_username,
up.display_name AS p_creator_display_name,
up.profile_photo_url AS p_creator_avatar,
COALESCE(up.is_verified, FALSE) AS p_creator_verified,

-- Business profile
ba.name AS p_business_name,
ba.logo_url AS p_business_logo,
COALESCE(ba.is_verified, FALSE) AS p_business_verified

FROM posts p
INNER JOIN post_media pm ON pm.post_id = p.id
LEFT JOIN user_profiles up ON up.id = p.user_id
LEFT JOIN business_accounts ba ON ba.id = p.actor_id AND p.actor_type = 'busi

-- Engagement counts (subquery to avoid GROUP BY on all columns)
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt FROM post_likes WHERE post_id = p.id
) lc ON TRUE
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt FROM post_comments WHERE post_id = p.id
) cc ON TRUE
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt FROM post_shares WHERE post_id = p.id
) sc ON TRUE

-- Relationships
LEFT JOIN my_friends mf ON mf.uid = p.user_id
LEFT JOIN my_follows mfl ON mfl.uid = p.user_id
LEFT JOIN my_business_follows mbf ON mbf.bid = p.actor_id AND p.actor_type =
LEFT JOIN blocked_users bu ON bu.uid = p.user_id

-- Current user interactions
LEFT JOIN post_likes ml ON ml.post_id = p.id AND ml.user_id = p_user_id

-- Review data
LEFT JOIN course_ratings cr ON cr.id = p.source_review_id
LEFT JOIN golf_courses gc ON gc.id = cr.course_id

WHERE p.status = 'published'
    AND bu.uid IS NULL -- Not blocked
    AND p.created_at > NOW() - INTERVAL '14 days' -- Max age
    AND (p_cursor IS NULL OR p.created_at < p_cursor) -- Cursor pagin
    AND NOT (p.id = ANY(p_seen_post_ids)) -- Skip already
    AND (
        -- Regular video posts: must have video under 3 min
        (p.source_review_id IS NULL AND pm.media_type = 'video'

```

```

        AND pm.duration_seconds IS NOT NULL AND pm.duration_seconds <= v_max_du
    OR
    -- Review posts: any media type allowed
    (p.source_review_id IS NOT NULL)
)

ORDER BY p.created_at DESC
LIMIT p_page_size * v_fetch_multiplier
),

-- Step 3: Score each candidate
scored AS (
    SELECT
        c.*,
        -- Engagement score
        (
            (c.p_like_count * 1.0)
            + (c.p_comment_count * 2.5)
            + (c.p_share_count * 3.0)
        )
        -- Recency decay: halves every 24 hours
        * POWER(0.5, EXTRACT(EPOCH FROM (NOW() - c.p_created_at)) / 3600 / v_recenc
        -- Social boost: friends 2x, following 1.5x, global 1x
        * CASE c.p_relation
            WHEN 'friend' THEN 2.0
            WHEN 'following' THEN 1.5
            ELSE 1.0
        END
    AS score,
    -- For creator diversity: rank within each creator
    ROW_NUMBER() OVER (
        PARTITION BY c.p_user_id
        ORDER BY
            (c.p_like_count * 1.0 + c.p_comment_count * 2.5 + c.p_share_count * 3.0
            * POWER(0.5, EXTRACT(EPOCH FROM (NOW() - c.p_created_at)) / 3600 / v_re
            DESC
        ) AS creator_rank
    FROM candidates c
),

-- Step 4: Apply creator diversity cap
filtered AS (
    SELECT * FROM scored
    WHERE creator_rank <= v_creator_cap
    ORDER BY score DESC
    LIMIT p_page_size
)

```

-- Step 5: Return results

SELECT

f.p\_id,  
f.p\_content,  
f.p\_created\_at,  
f.p\_user\_id,  
f.p\_actor\_type,  
f.p\_actor\_id,  
f.p\_status,  
f.p\_source\_review\_id,

f.pm\_id,  
f.pm\_media\_type,  
f.pm\_media\_url,  
f.pm\_poster\_url,  
f.pm\_stream\_id,  
f.pm\_duration,  
f.pm\_width,  
f.pm\_height,  
f.pm\_display\_order,

f.p\_creator\_username,  
f.p\_creator\_display\_name,  
f.p\_creator\_avatar,  
f.p\_creator\_verified,

f.p\_business\_name,  
f.p\_business\_logo,  
f.p\_business\_verified,

f.p\_like\_count,  
f.p\_comment\_count,  
f.p\_share\_count,

f.p\_review\_rating,  
f.p\_review\_course\_id,  
f.p\_review\_course\_name,  
f.p\_review\_course\_image,

f.p\_relation,  
f.p\_liked\_by\_me,  
f.p\_followed\_by\_me,

f.score

FROM filtered f

ORDER BY f.score DESC;



```
END;  
$$;
```

## Algorithm Breakdown

The scoring formula in plain English:

```
engagement_score = (likes × 1.0) + (comments × 2.5) + (shares × 3.0)  
                  × recency_decay(0.5 ^ (hours_old / 24))  
                  × social_boost(friend=2x, following=1.5x, global=1x)
```

### Why these weights:

- **Comments × 2.5** — A comment requires more effort than a like. It signals genuine interest.
- **Shares × 3.0** — Sharing means someone found it worth passing on. Strongest signal.
- **Likes × 1.0** — Baseline engagement. Easy to give, but still meaningful.
- **Recency decay at 24h half-life** — A post keeps half its score after 1 day, 25% after 2 days, ~6% after 4 days. This keeps the feed fresh without killing older viral content.
- **Social boost** — Friends' content gets 2x weight because Clbhouz is a social-first platform. You want to see your golf buddies' posts.
- **Creator cap at 2** — No single creator can dominate a page. Even if someone posts 10 times, you see their top 2.

**What about zero-engagement posts from new creators?** A brand new post (0 likes, 0 comments, 0 shares) gets a base score of 0. But because we fetch 3x the page size and fill from top scores down, new posts will still appear when there aren't enough high-scoring alternatives. In a small-community phase (early Clbhouz), this means most posts surface. As the community grows, engagement becomes the differentiator.

To prevent cold-start invisibility, new posts (< 1 hour old) get a minimum floor score:

Add this to the score calculation:

```
GREATEST(  
  /* engagement score formula */,  
  /* floor: 1.0 for posts < 1 hour old, decaying to 0 */  
  CASE WHEN c.p_created_at > NOW() - INTERVAL '1 hour'  
    THEN 1.0 * POWER(0.5, EXTRACT(EPOCH FROM (NOW() - c.p_created_at)) / 3600)  
    ELSE 0
```

```
END
)
```

This ensures every new post gets at least one hour of visibility in the feed before engagement determines its fate.

---

## **get\_friends\_feed — Postgres Function**

Simpler query for the Friends tab. No engagement scoring — pure chronological from people you follow.

```
CREATE OR REPLACE FUNCTION get_friends_feed(
  p_user_id UUID,
  p_page_size INT DEFAULT 10,
  p_cursor TIMESTAMPTZ DEFAULT NULL,
  p_seen_post_ids UUID[] DEFAULT '{}'
)
RETURNS TABLE (
  -- Same column signature as get_suggested_feed
  post_id UUID,
  post_content TEXT,
  post_created_at TIMESTAMPTZ,
  post_user_id UUID,
  post_actor_type TEXT,
  post_actor_id UUID,
  post_status TEXT,
  source_review_id UUID,
  media_id UUID,
  media_type TEXT,
  media_url TEXT,
  poster_url TEXT,
  stream_id TEXT,
  duration_seconds NUMERIC,
  width INT,
  height INT,
  display_order INT,
  creator_username TEXT,
  creator_display_name TEXT,
  creator_avatar_url TEXT,
  creator_is_verified BOOLEAN,
  business_name TEXT,
  business_logo_url TEXT,
  business_is_verified BOOLEAN,
  like_count BIGINT,
```

```

comment_count BIGINT,
share_count BIGINT,
review_rating NUMERIC,
review_course_id UUID,
review_course_name TEXT,
review_course_image TEXT,
creator_relation TEXT,
is_liked_by_me BOOLEAN,
is_followed_by_me BOOLEAN,
engagement_score NUMERIC
)
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
BEGIN
    RETURN QUERY
    WITH
    my_follows AS (
        SELECT following_id AS uid FROM user_follows
        WHERE follower_id = p_user_id
    ),
    my_business_follows AS (
        SELECT business_id AS bid FROM business_follows
        WHERE follower_id = p_user_id
    ),
    my_friends AS (
        SELECT friend_id AS uid FROM user_friends
        WHERE user_id = p_user_id AND status = 'accepted'
        UNION
        SELECT user_id AS uid FROM user_friends
        WHERE friend_id = p_user_id AND status = 'accepted'
    ),
    blocked_users AS (
        SELECT blocked_user_id AS uid FROM user_blocks
        WHERE blocker_user_id = p_user_id
        UNION
        SELECT blocker_user_id AS uid FROM user_blocks
        WHERE blocked_user_id = p_user_id
    )

    SELECT
        p.id,
        p.content,
        p.created_at,
        p.user_id,
        p.actor_type,
        p.actor_id,

```

```

p.status,
p.source_review_id,

pm.id,
pm.media_type,
pm.media_url,
pm.poster_url,
pm.stream_id,
pm.duration_seconds,
pm.width,
pm.height,
pm.display_order,

up.username,
up.display_name,
up.profile_photo_url,
COALESCE(up.is_verified, FALSE),

ba.name,
ba.logo_url,
COALESCE(ba.is_verified, FALSE),

COALESCE(lc.cnt, 0),
COALESCE(cc.cnt, 0),
COALESCE(sc.cnt, 0),

cr.rating,
gc.id,
gc.name,
gc.image_url,

CASE
    WHEN mfr.uid IS NOT NULL THEN 'friend'
    ELSE 'following'
END,

CASE WHEN ml.post_id IS NOT NULL THEN TRUE ELSE FALSE END,
TRUE, -- Always true in friends feed (by definition, we follow them)

0::NUMERIC -- No engagement scoring in friends feed

FROM posts p
INNER JOIN post_media pm ON pm.post_id = p.id
LEFT JOIN user_profiles up ON up.id = p.user_id
LEFT JOIN business_accounts ba ON ba.id = p.actor_id AND p.actor_type = 'busine

LEFT JOIN LATERAL (SELECT COUNT(*) AS cnt FROM post_likes WHERE post_id = p.id)

```

```

LEFT JOIN LATERAL (SELECT COUNT(*) AS cnt FROM post_comments WHERE post_id = p.
LEFT JOIN LATERAL (SELECT COUNT(*) AS cnt FROM post_shares WHERE post_id = p.id

LEFT JOIN my_friends mfr ON mfr.uid = p.user_id
LEFT JOIN blocked_users bu ON bu.uid = p.user_id
LEFT JOIN post_likes ml ON ml.post_id = p.id AND ml.user_id = p_user_id
LEFT JOIN course_ratings cr ON cr.id = p.source_review_id
LEFT JOIN golf_courses gc ON gc.id = cr.course_id

WHERE p.status = 'published'
  AND bu.uid IS NULL
  AND (p_cursor IS NULL OR p.created_at < p_cursor)
  AND NOT (p.id = ANY(p_seen_post_ids))
  AND (
    -- Personal follows
    p.user_id IN (SELECT uid FROM my_follows)
    OR
    -- Business follows
    (p.actor_type = 'business' AND p.actor_id IN (SELECT bid FROM my_business_f
  )
  AND (
    -- Video posts: under 3 min
    (p.source_review_id IS NULL AND pm.media_type = 'video'
      AND pm.duration_seconds IS NOT NULL AND pm.duration_seconds <= 180)
    OR
    -- Review posts: any media
    (p.source_review_id IS NOT NULL)
  )
)

ORDER BY p.created_at DESC
LIMIT p_page_size;

END;
$$;

```

---

## Section 3: Review Interleaving (Client-Side)

The RPC functions return posts in scored order (Suggested) or chronological order (Friends). Review interleaving happens client-side after the fetch, keeping the algorithm transparent and tuneable without DB migrations.

```

import { FeedPost, FeedTab } from '../types/media';

/** Algorithm constants */
const REVIEW_INTERVAL_SUGGESTED = 5; // Every 5th post is a review in Suggested
const REVIEW_INTERVAL_FRIENDS = 8;    // Every 8th post is a review in Friends

/**
 * Interleave review posts into the feed at fixed intervals.
 *
 * Takes a raw list of posts (mixed regular + reviews, already scored/sorted
 * from the RPC) and rearranges so reviews appear at predictable positions.
 *
 * Suggested: 2 reviews per 10 posts (positions 5, 10, 15, 20...)
 * Friends: ~1 review per 8 posts (positions 8, 16, 24...)
 *
 * If not enough reviews exist, the slot is filled with the next regular post.
 * If too many reviews exist, extras are dropped (they'll appear on the next page
 */
export function interleaveReviews(
  posts: FeedPost[],
  tab: FeedTab
): FeedPost[] {
  const interval = tab === 'suggested'
    ? REVIEW_INTERVAL_SUGGESTED
    : REVIEW_INTERVAL_FRIENDS;

  const reviews: FeedPost[] = [];
  const regular: FeedPost[] = [];

  // Separate reviews from regular posts
  for (const post of posts) {
    if (post.isReview) {
      reviews.push(post);
    } else {
      regular.push(post);
    }
  }

  // Build interleaved feed
  const result: FeedPost[] = [];
  let reviewIdx = 0;
  let regularIdx = 0;
  let position = 1;

  const totalTarget = posts.length;

  while (result.length < totalTarget && (regularIdx < regular.length || reviewIdx

```

```

    if (position % interval === 0 && reviewIdx < reviews.length) {
      // Review slot
      result.push(reviews[reviewIdx++]);
    } else if (regularIdx < regular.length) {
      // Regular slot
      result.push(regular[regularIdx++]);
    } else if (reviewIdx < reviews.length) {
      // No more regular posts – fill with reviews
      result.push(reviews[reviewIdx++]);
    }
    position++;
  }

  return result;
}

/**
 * Deduplicate posts by ID (safety net for pagination overlap).
 */
export function deduplicatePosts(posts: FeedPost[]): FeedPost[] {
  const seen = new Set<string>();
  return posts.filter((post) => {
    if (seen.has(post.id)) return false;
    seen.add(post.id);
    return true;
  });
}

```

---

## Section 4: Data Fetching Hooks

### hooks/useSuggestedFeed.ts — New File

```

import { useInfiniteQuery } from '@tanstack/react-query';
import { useRef, useCallback } from 'react';
import { supabase } from '@integrations/supabase/client';
import { FeedPost } from '../types/media';
import { interleaveReviews, deduplicatePosts } from '../utils/feedAlgorithm';
import { mapRowToFeedPost } from '../utils/feedMapper';

const PAGE_SIZE = 10;

async function fetchSuggestedPage(

```

```

    userId: string,
    cursor: string | null,
    seenPostIds: string[]
  ): Promise<{ posts: FeedPost[]; nextCursor: string | null }> {

    const { data, error } = await supabase.rpc('get_suggested_feed', {
      p_user_id: userId,
      p_page_size: PAGE_SIZE,
      p_cursor: cursor,
      p_seen_post_ids: seenPostIds,
    });

    if (error) {
      console.error('[SuggestedFeed] RPC error:', error);
      return { posts: [], nextCursor: null };
    }

    if (!data || data.length === 0) {
      return { posts: [], nextCursor: null };
    }

    // Map DB rows to FeedPost
    const posts = (data as any[]).map(mapRowToFeedPost);

    // Interleave reviews
    const interleaved = interleaveReviews(posts, 'suggested');

    // Next cursor = created_at of the last row result (before interleaving)
    const lastRow = data[data.length - 1] as any;
    const nextCursor = data.length >= PAGE_SIZE ? lastRow.post_created_at : null;

    return { posts: interleaved, nextCursor };
  }

export function useSuggestedFeed(userId: string | undefined) {
  const seenPostIds = useRef<string[]>([]);

  const query = useInfiniteQuery({
    queryKey: ['media-feed', 'suggested', userId],
    queryFn: async ({ pageParam }) => {
      if (!userId) return { posts: [], nextCursor: null };
      const result = await fetchSuggestedPage(
        userId,
        pageParam as string | null,
        seenPostIds.current
      );
    },
    // Track seen posts
  });

```



```

    for (const post of result.posts) {
      if (!seenPostIds.current.includes(post.id)) {
        seenPostIds.current.push(post.id);
      }
    }
    return result;
  },
  getNextPageParam: (lastPage) => lastPage.nextCursor,
  initialPageParam: null as string | null,
  enabled: !!userId,
  staleTime: 2 * 60 * 1000,    // 2 min before stale
  gcTime: 10 * 60 * 1000,      // 10 min garbage collection
});

const allPosts = deduplicatePosts(
  query.data?.pages.flatMap((p) => p.posts) ?? []
);

const resetSeen = useCallback(() => {
  seenPostIds.current = [];
}, []);

return {
  posts: allPosts,
  isLoading: query.isLoading,
  hasNextPage: query.hasNextPage,
  isFetchingNextPage: query.isFetchingNextPage,
  fetchNextPage: query.fetchNextPage,
  refetch: query.refetch,
  isRefetching: query.isRefetching,
  resetSeen,
};
}

```

## hooks/useFriendsFeed.ts — New File

```

import { useInfiniteQuery } from '@tanstack/react-query';
import { useRef, useCallback } from 'react';
import { supabase } from '@integrations/supabase/client';
import { FeedPost } from '../types/media';
import { interleaveReviews, deduplicatePosts } from '../utils/feedAlgorithm';
import { mapRowToFeedPost } from '../utils/feedMapper';

const PAGE_SIZE = 10;

```

```

async function fetchFriendsPage(
  userId: string,
  cursor: string | null,
  seenPostIds: string[]
): Promise<{ posts: FeedPost[]; nextCursor: string | null }> {

  const { data, error } = await supabase.rpc('get_friends_feed', {
    p_user_id: userId,
    p_page_size: PAGE_SIZE,
    p_cursor: cursor,
    p_seen_post_ids: seenPostIds,
  });

  if (error) {
    console.error('[FriendsFeed] RPC error:', error);
    return { posts: [], nextCursor: null };
  }

  if (!data || data.length === 0) {
    return { posts: [], nextCursor: null };
  }

  const posts = (data as any[]).map(mapRowToFeedPost);
  const interleaved = interleaveReviews(posts, 'friends');

  const lastRow = data[data.length - 1] as any;
  const nextCursor = data.length >= PAGE_SIZE ? lastRow.post_created_at : null;

  return { posts: interleaved, nextCursor };
}

export function useFriendsFeed(userId: string | undefined) {
  const seenPostIds = useRef<string[]>([]);

  const query = useInfiniteQuery({
    queryKey: ['media-feed', 'friends', userId],
    queryFn: async ({ pageParam }) => {
      if (!userId) return { posts: [], nextCursor: null };
      const result = await fetchFriendsPage(
        userId,
        pageParam as string | null,
        seenPostIds.current
      );
      for (const post of result.posts) {
        if (!seenPostIds.current.includes(post.id)) {
          seenPostIds.current.push(post.id);
        }
      }
    }
  });
}

```

```

    }
    return result;
  },
  getNextPageParam: (lastPage) => lastPage.nextCursor,
  initialPageParam: null as string | null,
  enabled: !!userId,
  staleTime: 1 * 60 * 1000,    // 1 min (friends content should feel fresh)
  gcTime: 10 * 60 * 1000,
});

const allPosts = deduplicatePosts(
  query.data?.pages.flatMap((p) => p.posts) ?? []
);

const resetSeen = useCallback(() => {
  seenPostIds.current = [];
}, []);

return {
  posts: allPosts,
  isLoading: query.isLoading,
  hasNextPage: query.hasNextPage,
  isFetchingNextPage: query.isFetchingNextPage,
  fetchNextPage: query.fetchNextPage,
  refetch: query.refetch,
  isRefetching: query.isRefetching,
  resetSeen,
};
}

```

---

## Section 5: Row-to-FeedPost Mapper

### `utils/feedMapper.ts` — New File

Single function that maps DB rows from EITHER RPC function to a `FeedPost`. Both RPCs return the same column signature, so one mapper works for both.

```

import { FeedPost, MediaItem, ReviewData, CreatorRelation } from '../types/media'

const CLOUDFLARE_STREAM_SUBDOMAIN = 'customer-[YOUR_SUBDOMAIN]';

function buildHlsUrl(streamId: string): string {

```

```

    return `https://${CLOUDFLARE_STREAM_SUBDOMAIN}.cloudflarestream.com/${streamId}
  }

function buildMp4Url(streamId: string): string {
  return `https://${CLOUDFLARE_STREAM_SUBDOMAIN}.cloudflarestream.com/${streamId}
}

function buildThumbnailUrl(streamId: string): string {
  return `https://${CLOUDFLARE_STREAM_SUBDOMAIN}.cloudflarestream.com/${streamId}
}

function extractStreamId(mediaUrl: string): string | null {
  if (!mediaUrl) return null;
  // Cloudflare Stream URLs: https://customer-xxx.cloudflarestream.com/{streamId}
  const match = mediaUrl.match(/cloudflarestream\.com\/([a-f0-9]+)/i);
  return match ? match[1] : null;
}

/**
 * Map a raw DB row from get_suggested_feed or get_friends_feed to a FeedPost.
 *
 * This is the ONLY place where DB column names are referenced.
 * Everything downstream works with the clean FeedPost type.
 */
export function mapRowToFeedPost(row: any): FeedPost {
  const streamId = row.stream_id || extractStreamId(row.media_url || '');
  const isReview = !!row.source_review_id;
  const isBusiness = row.post_actor_type === 'business';

  // Build media item
  const mediaItem: MediaItem = {
    id: row.media_id,
    type: row.media_type === 'video' ? 'video' : 'image',
    hlsUrl: streamId ? buildHlsUrl(streamId) : undefined,
    mp4Url: streamId ? buildMp4Url(streamId) : undefined,
    thumbnailUrl: row.poster_url || (streamId ? buildThumbnailUrl(streamId) : undefined),
    imageUrl: row.media_type === 'image' ? row.media_url : undefined,
    width: row.width || 1080,
    height: row.height || 1920,
    duration: row.duration_seconds ? Number(row.duration_seconds) : undefined,
  };

  // Build review data if applicable
  let review: ReviewData | null = null;
  if (isReview && row.review_course_id) {
    review = {
      reviewId: row.source_review_id,

```

```

        courseId: row.review_course_id,
        courseName: row.review_course_name || 'Unknown Course',
        courseImageUrl: row.review_course_image || null,
        rating: Number(row.review_rating) || 0,
    };
}

return {
    id: row.post_id,
    userId: row.post_user_id,
    actorType: (row.post_actor_type || 'personal') as 'personal' | 'business',
    actorId: row.post_actor_id || row.post_user_id,

    // Creator info – use business profile when actor is business
    username: isBusiness
        ? (row.business_name || row.creator_username || '')
        : (row.creator_username || ''),
    displayName: isBusiness
        ? (row.business_name || row.creator_display_name || '')
        : (row.creator_display_name || row.creator_username || ''),
    avatarUrl: isBusiness
        ? (row.business_logo_url || '')
        : (row.creator_avatar_url || ''),
    isVerified: isBusiness
        ? !!row.business_is_verified
        : !!row.creator_is_verified,
    creatorRelation: (row.creator_relation || 'none') as CreatorRelation,

    // Content
    caption: row.post_content || '',
    mediaItems: [mediaItem],
    createdAt: row.post_created_at,

    // Engagement – real counts, zero Math.random()
    likeCount: Number(row.like_count) || 0,
    commentCount: Number(row.comment_count) || 0,
    shareCount: Number(row.share_count) || 0,

    // Review
    review,
    isReview,

    // Interaction state
    isLikedByMe: !!row.is_liked_by_me,
    isFollowedByMe: !!row.is_followed_by_me,
};
}

```

**Note on multi-media posts:** The RPC returns one row per media item. Posts with multiple media items (carousels) will produce multiple rows with the same `post_id`. The mapper currently creates one `FeedPost` per row with a single media item. To support carousels, add a post-processing step in the fetch function:

```
// After mapping, group by post_id and merge mediaItems:
function groupMultiMedia(posts: FeedPost[]): FeedPost[] {
  const map = new Map<string, FeedPost>();
  for (const post of posts) {
    if (map.has(post.id)) {
      // Add media item to existing post
      map.get(post.id)!.mediaItems.push(...post.mediaItems);
    } else {
      map.set(post.id, { ...post });
    }
  }
  // Sort media items by display_order within each post
  for (const post of map.values()) {
    post.mediaItems.sort((a, b) => {
      const aOrder = (a as any).displayOrder ?? 0;
      const bOrder = (b as any).displayOrder ?? 0;
      return aOrder - bOrder;
    });
  }
  return Array.from(map.values());
}
```

Add `displayOrder` to `MediaItem` type:

```
export interface MediaItem {
  // ... existing fields ...
  displayOrder?: number; // For multi-media sorting
}
```

And in the mapper, set `displayOrder: row.display_order || 0` on each media item.

Call `groupMultiMedia` in the fetch functions before `interleaveReviews`:

```
const posts = groupMultiMedia((data as any[]).map(mapRowToFeedPost));
const interleaved = interleaveReviews(posts, 'suggested');
```

---

## Section 6: Tab System in FullscreenMediaViewer

### Rewrite `FullscreenMediaViewer.tsx`

The viewer now manages two tabs with independent feed states. Tab switching preserves scroll position and doesn't re-fetch.

```
import React, { useState, useCallback, useMemo } from 'react';
import { VideoPoolProvider } from '../VideoPoolProvider';
import { FeedContainer } from '../FeedContainer';
import { usePreloader } from '../hooks/usePreloader';
import { useSuggestedFeed } from '../hooks/useSuggestedFeed';
import { useFriendsFeed } from '../hooks/useFriendsFeed';
import { FeedPost, FeedTab } from '../types/media';
import { LoadingSkeleton } from '../LoadingSkeleton';
import { FeedTabToggle } from '../FeedTabToggle';

// Get the current user ID from your auth context
// Adjust this import to match your actual auth setup:
import { useAuth } from '@hooks/useAuth';

function FeedWithPreloader({
  posts,
  onNearEnd,
  onRefresh,
  isRefreshing,
  hasNextPage,
}: {
  posts: FeedPost[];
  onNearEnd: () => void;
  onRefresh: () => Promise<void>;
  isRefreshing: boolean;
  hasNextPage: boolean;
}) {
  usePreloader(posts);
  return (
    <FeedContainer
      posts={posts}
      onNearEnd={onNearEnd}
      onRefresh={onRefresh}
      isRefreshing={isRefreshing}
      hasNextPage={hasNextPage}
    />
  );
}
```

```

export default function FullscreenMediaViewer() {
  const { user } = useAuth();
  const userId = user?.id;

  const [activeTab, setActiveTab] = useState<FeedTab>('suggested');

  // Both feeds are always mounted – switching tabs is instant
  const suggested = useSuggestedFeed(userId);
  const friends = useFriendsFeed(userId);

  // Select active feed based on tab
  const activeFeed = activeTab === 'suggested' ? suggested : friends;

  const handleNearEnd = useCallback(() => {
    if (activeFeed.hasNextPage && !activeFeed.isFetchingNextPage) {
      activeFeed.fetchNextPage();
    }
  }, [activeFeed]);

  const handleRefresh = useCallback(async () => {
    activeFeed.resetSeen();
    await activeFeed.refetch();
  }, [activeFeed]);

  const handleTabChange = useCallback((tab: FeedTab) => {
    setActiveTab(tab);
    // Reset the media player's active index to 0 when switching tabs
    // The FeedContainer will handle scroll position
  }, []);

  if (activeFeed.isLoading) {
    return (
      <div style={{ width: '100%', height: '100dvh', background: '#000' }}>
        <FeedTabToggle activeTab={activeTab} onTabChange={handleTabChange} />
        <LoadingSkeleton />
      </div>
    );
  }

  if (!activeFeed.posts.length) {
    return (
      <div style={{
        width: '100%', height: '100dvh', background: '#000',
        display: 'flex', flexDirection: 'column', alignItems: 'center', justifyCo
        color: 'rgba(255,255,255,0.6)', fontFamily: '-apple-system, sans-serif',
      }}>

```



```

        <FeedTabToggle activeTab={activeTab} onTabChange={handleTabChange} />
        <p style={{ fontSize: 16 }}>
          {activeTab === 'friends'
            ? 'No posts from people you follow yet'
            : 'No posts to show'}
        </p>
      </div>
    );
  }

  return (
    <VideoPoolProvider>
      <div style={{ position: 'relative', width: '100%', height: '100dvh', backgr
        /* Tab toggle - positioned above the feed */
        <FeedTabToggle activeTab={activeTab} onTabChange={handleTabChange} />

        { /* Feed */ }
        <FeedWithPreloader
          posts={activeFeed.posts}
          onNearEnd={handleNearEnd}
          onRefresh={handleRefresh}
          isRefreshing={activeFeed.isRefetching}
          hasNextPage={activeFeed.hasNextPage ?? true}
        />
      </div>
    </VideoPoolProvider>
  );
}

```

---

## Section 7: Tab Toggle Component

### FeedTabToggle.tsx — New File

```

import React from 'react';
import { FeedTab } from '../types/media';

interface FeedTabToggleProps {
  activeTab: FeedTab;
  onTabChange: (tab: FeedTab) => void;
}

/**

```

```

* Pill-style tab toggle for Suggested / Friends.
* Positioned in the safe area at the top of the screen,
* floating above the video content.
*/
export function FeedTabToggle({ activeTab, onTabChange }: FeedTabToggleProps) {
  return (
    <div
      style={{
        position: 'absolute',
        top: 'max(env(safe-area-inset-top, 0px), 47px)',
        left: 0,
        right: 0,
        zIndex: 30,
        display: 'flex',
        justifyContent: 'center',
        pointerEvents: 'none',
      }}
    >
      <div
        style={{
          display: 'flex',
          gap: 0,
          background: 'rgba(0,0,0,0.35)',
          borderRadius: 20,
          padding: 3,
          pointerEvents: 'auto',
        }}
      >
        <TabButton
          label="Suggested"
          isActive={activeTab === 'suggested'}
          onClick={() => onTabChange('suggested')}
        />
        <TabButton
          label="Friends"
          isActive={activeTab === 'friends'}
          onClick={() => onTabChange('friends')}
        />
      </div>
    </div>
  );
}

```

```

function TabButton({
  label,
  isActive,
  onClick,

```

```

}: {
  label: string;
  isActive: boolean;
  onClick: () => void;
}) {
  return (
    <button
      onClick={e => {
        e.stopPropagation();
        onClick();
      }}
      style={{
        padding: '6px 18px',
        borderRadius: 17,
        border: 'none',
        background: isActive ? 'rgba(255,255,255,0.2)' : 'transparent',
        color: isActive ? '#FFFFFF' : 'rgba(255,255,255,0.55)',
        fontSize: 14,
        fontWeight: isActive ? 600 : 500,
        fontFamily: '-apple-system, BlinkMacSystemFont, sans-serif',
        cursor: 'pointer',
        transition: 'all 200ms ease',
        WebkitTapHighlightColor: 'transparent',
      }}
    >
      {label}
    </button>
  );
}

```

Design follows the Clbhouz glass morphism pattern from Tours Overview: semi-transparent dark background, white text, pill shape, subtle active state. Uses the canonical safe-area-inset-top pattern.

---

## Section 8: Supabase Table Check

The RPC functions reference these tables. Verify they exist (and create `post_shares` and `user_blocks` if they don't):

### Likely Existing

- `posts` — Core content table

- `post_media` — Media attachments
- `post_likes` — Like records (`user_id`, `post_id`)
- `post_comments` — Comment records
- `user_profiles` — Profile data
- `user_follows` — One-way follows (`follower_id`, `following_id`)
- `user_friends` — Mutual friends (`user_id`, `friend_id`, `status`)
- `business_follows` — Business follows (`follower_id`, `business_id`)
- `business_accounts` — Business profiles
- `course_ratings` — Review data (`id`, `course_id`, `rating`)
- `golf_courses` — Course info (`id`, `name`, `image_url`)

## Possibly Missing — Check and Create If Needed

**`post_shares`** — The RPC references this for share count. If it doesn't exist:

```
CREATE TABLE IF NOT EXISTS post_shares (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  post_id UUID REFERENCES posts(id) ON DELETE CASCADE,
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(post_id, user_id)
);
```

If you don't have a shares feature yet, create the table empty. The RPC will just return 0 for `share_count` until shares are implemented. Alternatively, remove the `share_count` lateral join from both RPCs and hardcode 0.

**`user_blocks`** — The RPC uses this for visibility filtering. If it doesn't exist:

```
CREATE TABLE IF NOT EXISTS user_blocks (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  blocker_user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  blocked_user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(blocker_user_id, blocked_user_id)
);
```

Same approach — create empty if missing. The filter will simply pass all users through.

**is\_verified on user\_profiles** — The RPC reads `up.is_verified`. If this column doesn't exist on `user_profiles`, add it:

```
ALTER TABLE user_profiles ADD COLUMN IF NOT EXISTS is_verified BOOLEAN DEFAULT FA
```

## Section 9: File Summary

### New Files (media-system/)

| File                                   | Purpose                                                              |
|----------------------------------------|----------------------------------------------------------------------|
| <code>types/media.ts</code>            | Updated with FeedTab, ReviewData, CreatorRelation, expanded FeedPost |
| <code>utils/feedAlgorithm.ts</code>    | Review interleaving + deduplication                                  |
| <code>utils/feedMapper.ts</code>       | DB row → FeedPost mapper (single source of truth)                    |
| <code>hooks/useSuggestedFeed.ts</code> | Suggested tab hook with engagement-scored RPC                        |
| <code>hooks/useFriendsFeed.ts</code>   | Friends tab hook with chronological RPC                              |
| <code>FeedTabToggle.tsx</code>         | Tab toggle UI component                                              |

### Modified Files

| File                                   | Change                                                      |
|----------------------------------------|-------------------------------------------------------------|
| <code>FullscreenMediaViewer.tsx</code> | Complete rewrite — tab system, dual feeds, auth integration |

### Supabase Migrations (SQL)

| Migration                                   | Purpose                                  |
|---------------------------------------------|------------------------------------------|
| <code>get_suggested_feed</code> function    | Engagement-scored feed with social graph |
| <code>get_friends_feed</code> function      | Chronological friends feed               |
| <code>post_shares</code> table (if missing) | Share tracking                           |

|                                              |                      |
|----------------------------------------------|----------------------|
| <code>user_blocks</code> table (if missing)  | Block filtering      |
| <code>is_verified</code> column (if missing) | Real verified status |

## NOT Modified (kept as-is)

All existing media-system components (FeedContainer, FeedItem, VideoPlayer, SocialOverlay, Scrubber, MediaCarousel, etc.) — they already handle `FeedPost` rendering. The new feed system only changes how posts are **fetchd and sorted**, not how they're **displayed**.

---

# Section 10: Implementation Order

## Step 1: Supabase Migrations

- Check for missing tables (`post_shares`, `user_blocks`, `is_verified` column)
- Create the `get_suggested_feed` RPC function
- Create the `get_friends_feed` RPC function
- Test both RPCs directly in Supabase SQL editor

## Step 2: Types + Utilities

- Update `types/media.ts` with new types
- Create `utils/feedMapper.ts`
- Create `utils/feedAlgorithm.ts`

## Step 3: Feed Hooks

- Create `hooks/useSuggestedFeed.ts`
- Create `hooks/useFriendsFeed.ts`
- Test both hooks with `console.log`

## Step 4: Tab System

- Create `FeedTabToggle.tsx`
- Rewrite `FullscreenMediaViewer.tsx`
- Wire tab switching

## Step 5: Test End-to-End

- Suggested tab loads engagement-scored posts
- Friends tab loads chronological followed-user posts
- Tab switching is instant (both feeds pre-loaded)
- Infinite scroll works on both tabs
- Pull-to-refresh resets seen posts and refetches
- Review posts appear at correct intervals
- No `Math.random()` anywhere — all data is real

Build each step completely before moving to the next. The Supabase RPCs are the foundation — everything else depends on them working.